Aula Extra E3 — NLP + Classificação (Aplicação)

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: Aplicação (notebook); AME §8.1.3 e Ap. A.2–A.3

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- descrever as etapas de limpeza e normalização de texto;
- construir a **matriz documento–termo** (*bag-of-words*) e explicar por que ela é esparsa;
- definir o **TF–IDF** e dizer o que ele corrige;
- escolher métodos adequados a alta dimensão esparsa e justificar a escolha;
- avaliar um classificador de *spam* com as métricas certas (Aula 08);
- montar o *pipeline* completo, sem vazamento (Aula 06).

Esta é a última aula do curso, e ela não traz nenhum método novo: traz um *problema* novo, e mostra que as ferramentas dos doze encontros anteriores já dão conta dele. A pergunta:

um modelo só entende números. Como transformar uma mensagem de texto em covariáveis?

1 O problema: filtrar spam

Usaremos a base `spam.csv` (SMS Spam Collection): 5 572 mensagens de SMS rotuladas como *ham* (legítima, $Y = 0$) ou *spam* ($Y = 1$), sendo 13,4% delas *spam*. É um problema de **classificação binária** (Aula 07) cujas covariáveis são *texto bruto* — e o desafio central é a *representação*.

Atenção

A base é **desbalanceada**. Como vimos na Aula 08, a acurácia engana aqui: um classificador que declarasse tudo *ham* já acertaria 86,6%. Precisaremos de precisão, *recall* e F_1. E os custos são assimétricos: bloquear uma mensagem legítima (falso positivo) costuma ser bem pior que deixar passar um *spam* — o que, pela Proposição 3.1 da Aula 08, significa exigir um corte *alto* para declarar *spam*.

2 Do texto aos atributos

2.1 Limpeza e normalização

Texto bruto é ruidoso. As etapas usuais de pré-processamento:

- *minusculizar* (`lower`): "Free" e "free" viram o mesmo *token*;
- remover **pontuação** e **números** (conforme o problema);
- *tokenizar*: quebrar a mensagem em palavras (*tokens*);
- remover ***stopwords***: palavras muito frequentes e pouco informativas ("the", "is", "and");
- *stemming/lematização* (opcional): reduzir flexões a um radical ("running" → "run").

Atenção

Cada uma dessas etapas *descarta informação*, e nem sempre a informação certa. Remover números destrói "ganhe R$ 1000", que é justamente um sinal de *spam*; minusculizar apaga "GRÁTIS" gritado em maiúsculas. Não aplique a lista por reflexo: cada decisão é um hiperparâmetro, e vale testá-la por validação cruzada como qualquer outro.

2.2 *Bag-of-words* e a matriz documento–termo

A representação mais simples é o ***bag-of-words*** (saco de palavras): ignora-se a ordem e conta-se a ocorrência de cada palavra. Constrói-se a **matriz documento–termo $\boldsymbol{X}$**, em que a linha i é a mensagem i, a coluna j é um termo do vocabulário, e x_{ij} é a contagem (ou presença binária) do termo j na mensagem i.

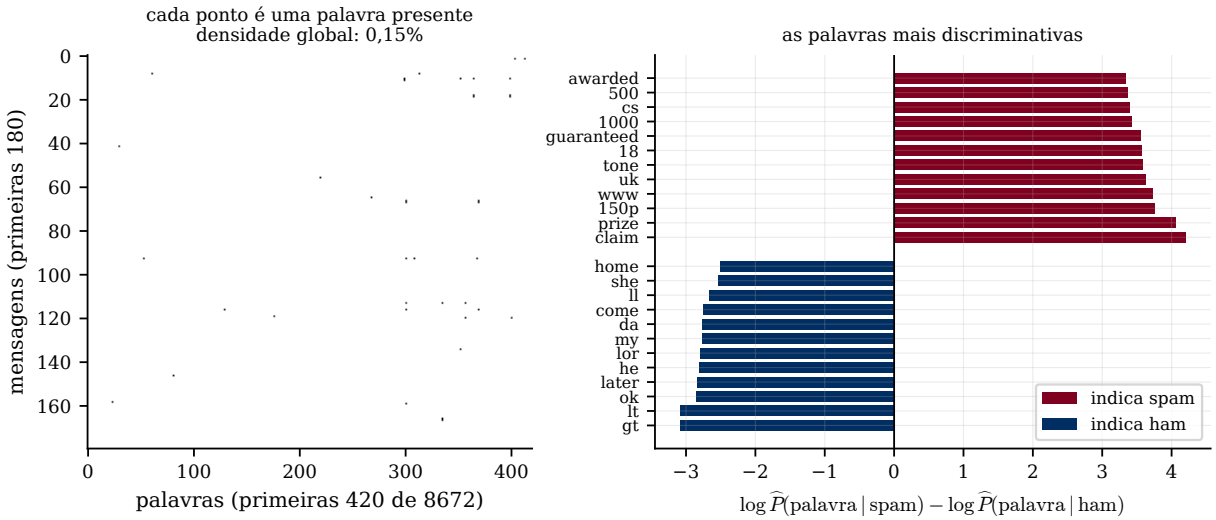


Figura 1: À esquerda, um pedaço da matriz documento–termo da base `spam.csv`: cada ponto preto é uma palavra presente numa mensagem. A matriz completa tem 5 572 linhas e 8 672 colunas, e apenas 0,15% das posições são não nulas — em média 13 palavras distintas por mensagem, de um vocabulário de quase nove mil. À direita, as palavras que mais deslocam a decisão de um Bayes ingênuo multinomial: de um lado "claim", "prize", "150p", "www"; do outro, o vocabulário do dia a dia.

Ideia central

Repare na desproporção que a figura registra: $5\,572 \times 8\,672$ dá quase 48 milhões de posições, das quais só umas 74 mil são não nulas. Guardar essa matriz densamente desperdiça mais de 99% da memória — daí os **formatos esparsos** (Apêndice A.3 do [AME]), que armazenam apenas as posições ocupadas. O `scikit-learn` devolve matrizes esparsas por padrão nos vetorizadores, e é por isso que o problema é tratável num laptop.

Atenção

Texto é o exemplo canônico de **alta dimensão + esparsidade**: milhares de palavras, mas a maioria irrelevante para a classe. Por isso métodos que *selecionam variáveis* (Lasso, Bayes ingênuo) vão bem, e o KNN vai mal — ele não seleciona, e a distância euclidiana entre duas mensagens é dominada pelas milhares de palavras que nenhuma das duas contém.

2.3 TF–IDF

Contagens cruas dão peso demais a palavras comuns. O **TF–IDF** (*term frequency–inverse document frequency*) reescala cada termo:

$$\text{tf-idf}(t,d) = \underbrace{\text{tf}(t,d)}_{\substack{\text{frequência do termo} \\ \text{no documento } d}} \times \underbrace{\log \frac{N}{\text{df}(t)}}_{\substack{\text{raridade do termo} \\ \text{na coleção}}},$$

onde $\text{df}(t)$ é o número de documentos que contêm t e N o total. Palavras que aparecem em *todo* documento têm IDF baixo (pouco discriminantes); palavras *raras e específicas* ganham peso.

Ideia central

O IDF é uma versão automática da remoção de *stopwords*, e melhor: em vez de usar uma lista fixa, ele descobre *nos seus dados* quais palavras são comuns demais para informar. Se aparece em todo documento, $\text{df}(t) = N$, o logaritmo zera e a palavra some sozinha. É o mesmo espírito do Lasso na Aula 02 — deixar os dados decidirem o que descartar.

3 Análise exploratória

Antes de modelar, vale *olhar* os dados: quais as palavras mais frequentes em *ham versus spam*? O painel direito da Figura 1 responde de forma quantitativa, ordenando as palavras por $\log \widehat{\mathbb{P}}(\text{palavra} \mid \text{spam}) - \log \widehat{\mathbb{P}}(\text{palavra} \mid \text{ham})$. Do lado do *spam* aparecem "claim", "prize", "guaranteed", "150p", "www", "uk", "1000" — o vocabulário de promoções e números de tarifação. Do lado do *ham*, palavras banais de conversa. Esse contraste já mostra que o *bag-of-words* carrega sinal suficiente para separar as classes, antes de ajustar qualquer modelo.

4 Classificando

Quais métodos usar? Os que prosperam em alta dimensão esparsa:

Bayes ingênuo (multinomial)
: o "cavalo de batalha" do texto. A suposição de independência condicional (Aula 07) é grosseira para palavras — "prize" e "win" claramente andam juntas — mas funciona surpreendentemente bem e é rápida. A versão *multinomial* modela contagens. (Lembre da Aula 07: trabalhe em escala logarítmica para evitar *underflow*.)

Logística penalizada (Lasso/Ridge)
: discriminativa, robusta, e o Lasso faz *seleção de palavras* — gerando um modelo interpretável (quais palavras pesam para *spam*). É o que o [AME] usa no exemplo das resenhas da Amazon.

Florestas / *boosting*
: também aplicáveis; XGBoost costuma ir bem.

Ideia central

Por que Bayes ingênuo combina tão bem com texto? Porque o modelo multinomial sobre contagens é natural, há pouquíssimos parâmetros por palavra (baixa variância) e a independência condicional, embora falsa, raramente troca a *classe* de maior probabilidade — e, para classificar, basta acertar o lado do corte.

Atenção

Mas atenção ao que a Aula 08 mediu: o Bayes ingênuo com covariáveis correlacionadas — e palavras *são* correlacionadas — produz probabilidades muito descalibradas, mesmo classificando bem. Se você for ajustar o corte por custos (como o problema de *spam* pede), estará mexendo justamente nos números em que não se pode confiar. Nesse caso, ou use a logística, ou calibre o Bayes ingênuo com `CalibratedClassifierCV`.

5 Avaliação

Como o problema é desbalanceado (Aula 08):

- reporte a **matriz de confusão**, **precisão**, ***recall*** e F_1 — não só a acurácia;
- para *spam*, a *precisão* sobre a classe "spam" é crítica: precisão alta significa que quase tudo que você bloqueou era mesmo lixo;
- ajuste o **corte** de decisão conforme os custos: se um falso positivo é dez vezes mais caro que um falso negativo, o corte ótimo é $10/11 \approx 0{,}91$, não 0,5.

Observação 5.1 (a métrica escolhida depois é a métrica errada)**.** Escolher a métrica *antes* soa como formalidade, e a §8 da `Aula prática E3` mostra que não é. Ela acrescenta ao saco de palavras duas colunas que ele não enxerga — o comprimento da mensagem e a proporção de dígitos —, e que separam muito: no treino, o *ham* médio tem 70,9 caracteres e 0,4% de dígitos, contra 138,8 e 11,7% do *spam*. O resultado:

	acurácia	AUC	AP
só texto	0,9821	0,9879	**0,9691**
texto + 2 colunas	0,9862	0,9950	**0,9572**

A acurácia sobe, a AUC sobe — e a **AP desce**. Não é contradição: são perguntas diferentes. A AUC melhora porque as duas colunas arrumam o *grosso* do ranking, marcando com mais confiança as mensagens curtas e sem dígitos. A AP olha a *cabeça* do ranking, onde o TF–IDF já ia muito bem, e ali acrescentar duas colunas densas ao lado de milhares de esparsas mexe nos pesos dos termos mais informativos.

Se o filtro vai bloquear as mensagens de maior escore — que é o caso — quem manda é a AP, e por ela essas duas colunas não pagam o que custam. Se a métrica tivesse sido escolhida depois de ver os números, as três linhas justificariam três decisões diferentes.

6 O *pipeline* completo (síntese do curso)

Tudo se encaixa num único *pipeline* (Aula 06): vetorização → classificador, com hiperparâmetros escolhidos por validação cruzada (Aula 03) *sem vazamento* — e o vetorizador deve aprender o vocabulário **só no treino**.

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix

sms = pd.read_csv("spam.csv", encoding="ISO-8859-1")
X, y = sms["text"], (sms["class"] == "spam").astype(int)
```

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          stratify=y, random_state=0)

# vetorizacao + classificador num unico objeto (sem vazamento!)
pipe = Pipeline([
    ("vec", TfidfVectorizer(lowercase=True, stop_words="english")),
    ("clf", MultinomialNB()),
])
grade = {"vec__ngram_range": [(1,1), (1,2)],
         "clf__alpha": [0.1, 0.5, 1.0]} # suavizacao de Laplace
busca = GridSearchCV(pipe, grade, cv=5, scoring="f1").fit(X_tr, y_tr)

y_hat = busca.predict(X_te)
print(confusion_matrix(y_te, y_hat))
print(classification_report(y_te, y_hat)) # precisao, recall, F1
```

Atenção: por que o vetorizador *tem* que estar no pipeline

Se você vetorizar tudo antes de separar treino e teste, o vocabulário — e os valores de IDF — serão calculados usando as mensagens de teste. É vazamento, mas da categoria **leve** da Aula 06, como padronizar ou codificar categóricas com o conjunto todo: o vetorizador aprende com as mensagens, mas *não olha os rótulos*, e por isso não consegue fabricar sinal a partir de ruído. Corrija assim mesmo, porque custa zero: dentro do *pipeline*, cada dobra reconstrói o vocabulário do zero, e palavras que só aparecem no teste são simplesmente ignoradas — que é exatamente o que aconteceria em produção.

Observação 6.1 (o que *não* vimos)**.** O *bag-of-words* joga fora a ordem das palavras: para ele, "não é spam" e "é spam não" são idênticos. Métodos modernos de NLP — *embeddings*, redes recorrentes, *transformers* — existem justamente para recuperar essa informação, e estão fora do escopo do curso ([ISLP] §10.4–10.5 dá uma introdução). Vale saber que, para muitas tarefas de classificação de texto, a diferença de desempenho entre um TF–IDF com logística e um modelo grande é menor do que se imagina — e o primeiro roda em segundos e você consegue explicar.

O notebook `Aula prática E3` percorre a limpeza, a exploração e a classificação na base `spam.csv`.

Resumo (e encerramento do curso)

- Texto vira número pelo *bag-of-words*: matriz documento–termo, gigantesca e **esparsa** — 0,15% de posições ocupadas na nossa base (Fig. 1).
- Limpeza e normalização descartam informação: trate as escolhas como hiperparâmetros.
- **TF–IDF** reescala pela raridade, descontando automaticamente as palavras comuns demais.
- Alta dimensão + esparsidade ⇒ Bayes ingênuo e logística penalizada; KNN vai mal.
- Avalie com precisão/*recall*/F_1 e ajuste o corte pelos custos (Aula 08); cuidado com a calibração do Bayes ingênuo.
- Tudo num *pipeline* (Aula 06): o vetorizador aprende o vocabulário *só* no treino.

E é assim que o curso fecha. Este problema não exigiu nenhuma ideia que você já não tivesse: a representação veio da Aula 06, o classificador veio da Aula 07, a avaliação veio da Aula 08 e a

montagem sem vazamento veio da Aula 03. O arcabouço da Aula 01 — risco, viés–variância, seleção de modelos — continua sendo o mesmo, quer as covariáveis sejam a temperatura crítica de um supercondutor ou as palavras de um SMS.

Leitura recomendada. [AME] §8.1.3 (Bayes ingênuo), §3.8 e §8.8 (os exemplos com as resenhas da Amazon, que são a versão em regressão e em classificação do que fizemos aqui), Apêndice A.2 (manipulação de textos) e A.3 (matrizes esparsas). [ISLP] §10.4 (*Document Classification*), para ver o mesmo problema tratado com redes neurais. Documentação do `scikit-learn`: *Working With Text Data*, que é um tutorial completo e curto.